SORTING STRATEGIES

(Alex Chan)

Contents

1. Spark Sorting Strategies and Behavior 3
1.1 Sort-Merge Join 3
1.2 Broadcast Hash Join (BHJ) 4
1.3 Shuffle Hash Join (SHJ) 5
1.4 Broadcast Nested Loop Join (BNLJ) 6
1.5 Cartesian Product Join 7
1.6 Comparison of Different Sorting Strategies 8
2. Recommended Strategy 9
APPENDIX: Adaptive Query Execution (AQE) 10
REFERENCES: 10

1. Spark Sorting Strategies and Behavior

Spark supports **five join strategies**, each with distinct sorting and shuffling characteristics:

1.1 Sort-Merge Join

Sort-Merge join is the default strategy for large-to-large table joins. During the joins, both datasets are shuffle (repartition) by the join key, then each partition is sorted by the join key independently.

There are 3 phases in Sort-Merge Join (SMJ)

1. **Shuffle** – both datasets are shuffled
2. **Sort** – records are sorted by key on both sides
3. **Merge** – iterate over both sides and join based on the join key

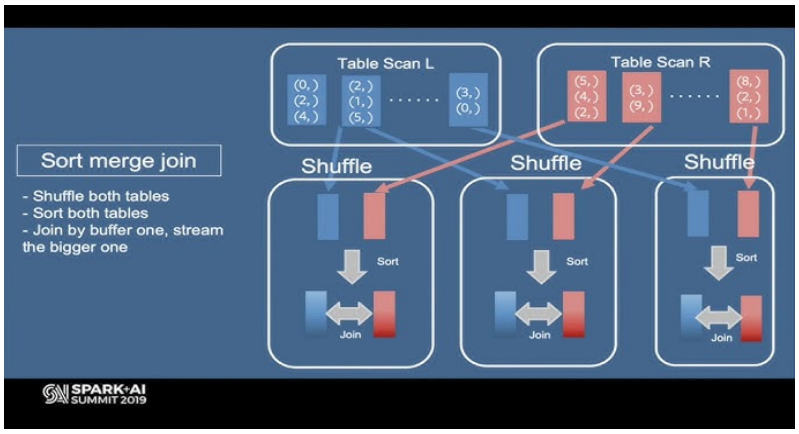

Illustration of Sort Merge Join

Configuration:

```
# Hint for sort-merge join
df1.join(df2.hint("MERGE"), "user_id")

# Aliases: SHUFFLE_MERGE, MERGEJOIN
# Prefer sort-merge join (default behavior)
spark.conf.set("spark.sql.join.preferSortMergeJoin", "true")
```

Pros	Cons
• **Handles large datasets** – works even partitions cannot fit in memory (spills to disk gracefully) • **Support all join types** – inner, left, right, full outer, etc	• **Sorting overhead** – expensive CPU operation, especially on large partitions • **Requires shuffle** - both sides must be redistributed across the network, thus incur large network I/O

Pros	Cons
• Skew friendly with AQE[1][2] – AQE can split skewed partitions during sort-merge joins (more details will discuss in later sections)	

Best for: Large to large table joins, especially when neither sides can fit into memory for broadcast. Spark selects this by default when broadcast thresholds are not met.

1.2 Broadcast Hash Join (BHJ)

Spark takes the smaller table, and broadcasts (copies) to every executor in the cluster to build an in-memory hash table on each executor. This is done by using Spark broadcast variable. The larger table stays put in the same partitioned, and each executor performs a local hash lookup for the join key or find matches.

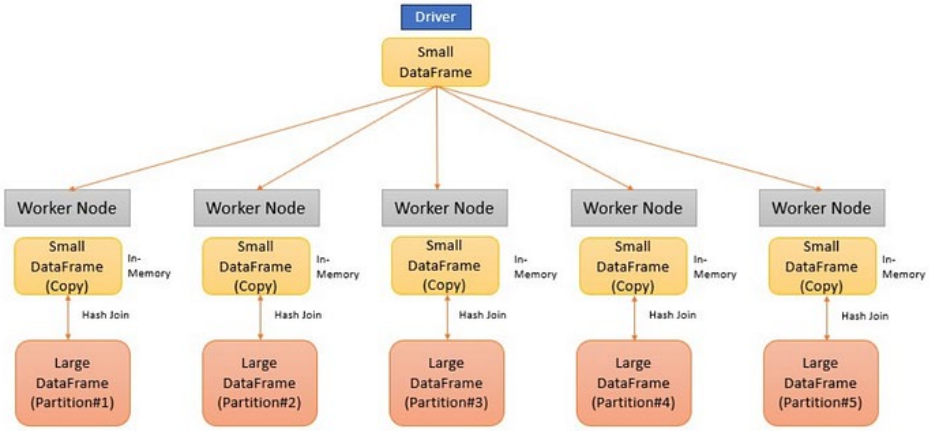

Illustration of Broadcast Hash Join

[1] In 2026, AQE may overrides this setting by convert SMJ to SHJ during runtime.
[2] AQE – Adaptive Query Execution (AQE)

Configuration:

```
# Hint for shuffle hash join
df1.join(df2.hint("SHUFFLE_HASH"), "user_id")

# Prefer shuffle hash over sort-merge globally
spark.conf.set("spark.sql.join.preferSortMergeJoin", "false")

# With AQE, convert to shuffle hash if partitions are small
spark.conf.set("spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold", "64MB")
```

Pros	Cons
• **No shuffle required** – the larger table is not moved • **Extremely fast** – local has lookups are O(1)	• **Memory risk** – executors may run out-of-memory if the broadcast table is too large • **Driver bottleneck**: depends on driver alone to collect and distributes to the small table

Best for: Joining a large Fact table with small dimensions (reference table)

1.3 Shuffle Hash Join (SHJ)

Both datasets are shuffled based on the join key into co-located partitions. For each partition pair, Spark builds an in-memory hash table from the smaller table of each partition, and probes it wit rows against the bigger table.

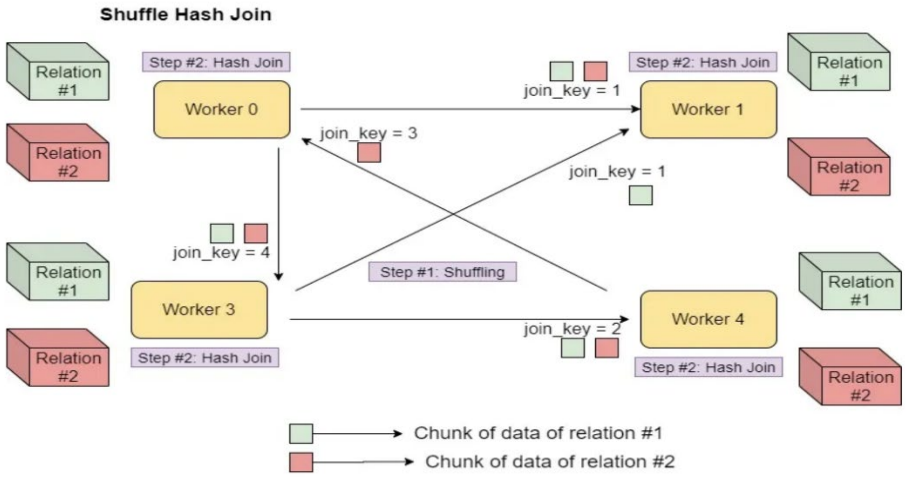

Illustration of Shuffle Hash Join

Configuration:

```
# Hint for shuffle hash join
df1.join(df2.hint("SHUFFLE_HASH"), "user_id")

# Prefer shuffle hash over sort-merge globally
spark.conf.set("spark.sql.join.preferSortMergeJoin", "false")

# With AQE, convert to shuffle hash if partitions are small
spark.conf.set("spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold", "64MB")
```

Pros	Cons
• **No sorting required** – faster than sort-merge join (SMJ) when partitions fit in memory. • **Fast**– Hash lookups are faster than merge scans	• **Requires shuffle** – network I/O and disk spill overhead • **Memory risk**: needs to build in-memory hash-table per partition running Out-of-Memory risks • **Vulnerable to data skew**: uneven key distributions may crate huge partitions

Best for: Medium-sized tables when one side is notably smaller and partitions fits comfortably in memory.

1.4 Broadcast Nested Loop Join (BNLJ)

The smaller dataset is broadcast to all executor. For each row in the large table, spark will iterate through every row in the broadcast dataset to evaluate if they match .

Configuration:

```
# Set the max size (in bytes) of a table that Spark will automatically broadcast.
# Default is 10MB (10485760 bytes). Increase this to force broadcasting of larger tables.
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "20971520") # 20MB

# Allow Cartesian Products (necessary if you are doing cross joins that trigger BNLJ)
spark.conf.set("spark.sql.crossJoin.enabled", "true")
```

Pros	Cons
• **Support Non-Equi joins** – SMJ and SHJ requires equality (e.g. A.id = B.id) They cannot handle inequalities such as (A.date > B.date and A.date < B.date). BNLJ is the only way Spark can proeess. • **Zero Network shuffle (For the large table):** The larger stays where it is. • **Universal Compatibility** – Can cater for data is skewed, unsorted or missing keys since it checks every row.	• **High Time Complexity** – Because the performance is O(MxN), the CPU time grows exponentially as the dataset grows. • **High Memory risk** - needs to build in-memory hash-table per partition running Out-of-Memory risks

Best for: used for Non-Equi Joins.

1.5 Cartesian Product Join

A Cartesian Product join is used when you want to join every single row of dataset A with every single row of dataset B. Example:

1. If one of the tables is small to fit in memory, Spark will broadcast the small table ands perform BNLJ.
2. However, if both tables are large, Spark will use Cartesian Product Join which shuffles the data across the cluster – makes it even slower than BNLJ.

Configuration:

```
# Allow Cartesian Products
spark.conf.set("spark.sql.crossJoin.enabled", "true")

# Hint for nested loop join
df1.join(df2.hint("SHUFFLE_REPLICATE_NL"), df1.id > df2.id)
```

Pros	Cons
• **Support Non-Equi joins** – SMJ and SHJ requires equality (e.g. A.id = B.id) They cannot handle inequalities such as (A.date > B.date and A.date < B.date). BNLJ is the only way Spark can proeess.	• **High Time Complexity** – Because the performance is O(MxN), the CPU time grows exponentially as the dataset grows. • **High Memory risk** - needs to build in-memory hash-table per partition running Out-of-Memory risks

Best for: Avoid unless necessary. If you need a Non-equi or cross join, ensure both sides are small.

1.6 Comparison of Different Sorting Strategies

Strategy	Shuffle	Sort	Best for
Broadcast Hash Join	None	None	Small-large joins
Sort-Merge Join	Both	Both	Large-large equi joins
Shuffle Hash Join	Both	None	Medium joins, memory permits
Broadcast Nested Loop	None	None	Non-equi joins, small table
Cartesian Product	Varies	None	Cross Joins (avoid if possible)

2. Recommended Strategy

- Dataset A: ~ 1 million rows
- Dataset B: ~10,000 rows
- Join Key: geographical_location_oid (equip-join)

Based on the table in Section 1.6, Broadcast Hash Join Strategy is recommended. This is because:

1. **Dataset B is very small** (~10,000 rows x 2 columns). Even with generous string lengths (varchar 500), the table would occupy well under 10 MB in memory — comfortably within Spark's default autoBroadcastJoinThreshold of 10 MB.

   Max. Dataset B size =500 x 8 x 10,000 = 5.08MB

2. **No shuffle is required**. Dataset A remains in place across its existing partitions. This eliminates the most expensive I/O operation in a distributed join.

3. **No sorting is required.** The hash lookup is O(1) per row, making the total join cost O(n) where n is the size of Dataset A.

4. **Only one scan of Dataset A is needed**. Each executor independently joins the broadcast hash map locally, achieving maximum parallelism with zero inter-executor communication.

5. **Broadcast Hash Join avoids all the costs of Sort-Merge Join**.

   - Shuffling both datasets by geographical_location_oid
   - Sorting both sides per partition (O(n log n))
   - Merging sorted streams

APPENDIX: Adaptive Query Execution (AQE)

Adaptive Query Execution is an optimization technique in Spark SQL (enables by default) that enhances performance by re-optimizing query plans based on runtime statistics.

It can dynamically adjust query execution plans such as optimizing shuffle partitions, converting join strategies and handling data skew as the query run.

Key benefits of AQE:

- **Dynamic Coalescing of Shuffle Partitions:** AQE reduces overhead by combining small, adjacent shuffle partitions into larger ones based on actual data size.
- **Dynamic Join Strategy Selection:** It can convert sort-merge joins into more efficient broadcast hash joins during runtime if one side of the join is small enough.
- **Dynamic Skew Join Handling:** AQE detects data skew and splits skewed partitions into smaller, more manageable sub-partitions, improving parallelism.
- **Performance Optimization:** By using real-time data metrics, it speeds up Spark SQL queries, often resulting in significant execution time reduction

For more information on AQE, please refer to Spark documentation
https://spark.apache.org/docs/latest/sql-performance-tuning.html#adaptive-query-execution

REFERENCES:

1. Different types of join strategies in apache spark - https://oindrila-chakraborty88.medium.com/different-types-of-join-strategies-in-apache-spark-5c0066999d0d
2. A Deep Dive into Skewed Joins, GroupBy Bottlenecks, and Smart Strategies to Keep Your Spark Jobs Flying - https://www.canadiandataguy.com/p/a-deep-dive-into-skewed-joins-groupby
3. Join Strategies - https://www.sparkplayground.com/tutorials/spark-theory/join-strategies
4. How does Shuffle Sort Merge Join work in Spark? - https://www.bigdatainrealworld.com/how-does-shuffle-sort-merge-join-work-in-spark/
5. Sort-Merge-Join in Spark - https://akash-dwivedi.medium.com/sort-merge-join-in-spark-9ebf40436bd3